

Contents

Constraint Optimization through RTL Enhancement Using Generative AI	1
0. Thesis	1
1. Deliverable 1 – RTL timing analysis framework	1
2. Deliverable 2 – GenAI-based RTL optimization engine	2
3. Deliverable 3 – Critical path and timing violation analysis	3
4. Deliverable 4 – Optimized RTL implementation	3
5. Deliverable 5 – Timing, frequency and PPA comparison	4
6. Deliverable 6 – Formal equivalence verification report	5
7. Deliverable 7 – Interactive demo	11
8. What we found in the benchmark that nobody was looking for	12
9. Design decisions and dead ends	12
10. Honest limitations	13

Constraint Optimization through RTL Enhancement Using Generative AI

Team: Timing Blinders · **College:** Indian Institute of Technology Bombay **Members:** Shubhanshu Shivhare, Nagarjun BV **Track:** Digital · **Astera Labs Nebula 2026**

0. Thesis

Every published LLM-for-timing-closure system we read builds the same loop: read a timing report, ask a model for an RTL rewrite, check the rewrite is equivalent, keep it if the slack improved. We built that loop. Then we spent most of the project on the part those systems do not address, which is this:

Equivalence is necessary and it is not sufficient.

An equivalence checker answers one question – do these two circuits compute the same function of their inputs? – and answers it well. Three classes of damage an LLM can do to RTL are invisible to it, and all three are reachable by the four transforms the problem statement allows:

- **Parent latency:** a pipelined module can be correct N cycles later while its parent still consumes the old cycle. **G1b** miters the parent over both children without a delay wrapper.
- **CDC structure:** removing a synchronizer stage can preserve the Boolean function and still fail in silicon. **G2** compares the crossing structure.
- **Generated clocks:** combinational and registered clock logic may agree at sampled edges while differing between them. **G2c** classifies the clock structure and rejects a regression.

None of these is hypothetical here. Each one either fired on this design or was found in it. Section 6 gives the evidence.

1. Deliverable 1 – RTL timing analysis framework

(status: complete)

`slicer.py` wraps OpenSTA against the placed-and-routed database and turns `report_checks` output into structured, per-module facts the rest of the flow can act on.

What it emits per critical path:

- slack, clock, stage count, total combinational delay
- delay **attributed to each module instance** on the path, as ns and as a share of the path
- start and end hierarchy chains
- whether the path touches a CDC structure
- the module worth editing, and every instance of it on the path

Two collapses keep the call budget sane. Twenty endpoints of the same FIFO word are one logical path, collapsed by signature. Two paths whose worst module is the same module are two symptoms of one edit, collapsed by target – and the record keeps how many paths each target was blamed for, so a module holding fifteen near-critical paths is visibly worth more than one holding a single path at the same slack.

Baseline, nebula_bench post-route: 20 reported paths collapse to 7 RTL targets. The selected `clk_s5` path has 195 logic stages, with 85.7% of its cell delay inside three chained `fp8_adder` instances. Parasitic-aware signoff ranks the failing domains as `clk_s5` –25.2539 ns, `clk_s8` –1.1998 ns, `clk1` –0.9670 ns, and `clk_s3` –0.1499 ns.

1.1 The lever selector

The lever column is the framework's own contribution, and it answers a question the problem statement implies but does not ask: *when is a timing violation an RTL problem at all?*

Not every violation is. A path missing by a few hundred picoseconds is going to be fixed by the synthesiser upsizing a cell, and spending a model call plus a formal proof on it duplicates work that happens for free and carries no equivalence risk. So each path is labelled:

- **none** – the path meets timing. Nothing to close.
- **gate** – it misses by less than an *optimistic* gate-level pass could recover. `GATE_HEADROOM = 0.20` of the path's own combinational delay – sizing, buffering, cell swaps, tighter placement. Deliberately the optimistic end of the published range for an open PDK, and sky130hd is single-Vt so there is no low-Vt escape hatch; erring high means the selector prefers the cheap lever and only escalates when even the optimistic bound falls short.
- **rtl** – the gap exceeds anything gate-level can reach, so the structure itself must change.

One veto overrides the arithmetic: a path under four stages goes back to **gate** however bad its slack, because a deep cell or a long wire has no structure to restructure.

On this design the split is stark. At the 12.5 ns target, signoff reports `clk_s5 WNS -25.2539 ns`; the measured period sweep closes at 38.98 ns. The gap is well outside the selector's optimistic gate-level reach, with 85.7% of the diagnostic path's cell delay inside three *chained* `fp8_adder` instances. That is the path the model is asked about.

Seven targets become one model call.

2. Deliverable 2 – GenAI-based RTL optimization engine

(status: complete)

`loop.py` runs: slice -> select -> propose -> validate -> G1 -> G1b -> G2 -> G2c -> accept -> G3.

Model access. The engine is provider-agnostic (`llm.py` speaks the OpenAI chat-completions shape; OpenRouter and Google Gemini are configured, and the model is a flag). The accepted design run used `gemini-3.5-flash`; the bake-off also exercises open-weight Nemotron models under the same targets and gate budget. Provider quotas affect whether a response is available, so every API response and outage is preserved rather than assuming a free key will always reproduce it on demand.

The model is constrained, not trusted. It returns JSON naming one of the five allowed transforms plus the complete rewritten module – never prose, never a diff. Anything unparseable is recorded as a refusal, which is a real outcome the loop handles, not a crash.

Every rejection is fed back with its evidence. This turned out to matter more than any prompt engineering. Our first counterexample feedback sent the model the first 800 characters of `sat`'s output table – of which seventeen of twenty-two lines were `init \gate.add_0 = 0`. The model repeated its mistake because it had never been told what the mistake was. `summarize()` now drops the reset rows, keeps the header, and puts the mismatched outputs first. The filtered table immediately exposed `in_rst = 1` at the divergence cycle, and the same model that had failed twice produced a correct four-stage pipeline on the next attempt.

Cost. The transform catalogue is byte-identical on every call and is the bulk of the tokens, so it is sent as its own cacheable system block where the provider supports it. The primary accepted run records **6 proposals**: two accepted, three formally disproved at G1, and one rejected structurally.

2.1 Same-task model bake-off

Each arm receives the same parasitic-aware targets, transform catalogue, retry budget, and gates. The corrected merged evidence is `artifacts/bakeoff_all_20260911/`.

licence	model	proposals	accepted	where the others stopped
open-weight	Nemotron Ultra 550B	7	1	validate 3, audit 1, G1 2
open-weight	Nemotron Super 120B	5	0	propose 2, validate 2, G1 1
open-weight	Nemotron Nano 30B	5	0	propose 2, validate 2, G1 1
open-weight	Gemma 4 31B	1	<i>not tested</i>	provider HTTP 429
free closed-weight	Gemini 3.5 Flash	6	1	propose 1, G1 4
free closed-weight	Gemini 3.1 Pro	1	<i>not tested</i>	provider quota

The Nemotron Ultra history originally reported 2/7. Its second “accept” targeted `fp4_dot_unit` but returned and proved another `fp8_adder` rewrite; the final tree changed only `fp8_adder.v`. The preserved record is now marked `audit`, giving the defensible result 1/7. Future runs reject a wrong primary module and a byte-identical rewrite before buying a formal proof. Gemma was

retried alone on 11 September after its live model slug passed the public probe; OpenRouter again returned HTTP 429 before any proposal reached a gate. Both untested rows are absence of evidence, not model failures.

3. Deliverable 3 – Critical path and timing violation analysis

(status: complete)

The benchmark is a 5-master-clock RISC-V SoC subsystem: five asynchronous master domains, a generated clock on every master, CDC across all five, clock dividers at four ratios, 113,036 cells post-synthesis.

Baseline timing, post-route:

Metric	Value
WNS	–25.2539 ns (clk_s5, 12.5 ns period)
TNS	–3384.4421 ns
Hold WNS	–0.1329 ns
Failing domains	4 of 11 reported (clk_s5, clk_s8, clk1, clk_s3)
DRC violations	0

Per-clock WNS:

clk1	10.0 ns	–0.9670	clk_s2	20.0 ns	+11.7365
clk_s1	25.0 ns	+18.5562	clk_s2_gate	20.0 ns	+17.9100
clk_s1_gate	25.0 ns	+23.8439	clk_s3	10.0 ns	–0.1499
clk_s4	40.0 ns	+32.9459	clk_s4_gate	40.0 ns	+38.1878
clk_s5	12.5 ns	–25.2539	clk_s5_gate	12.5 ns	+10.5825
clk_s8	10.0 ns	–1.1998			

Root cause of the one failure. clk_s5 clocks axi_lite_dot, an AXI-Lite peripheral computing a 4-element FP8 dot product. Its worst path is 195 logic stages: eight parallel fp4_mul, then a *combinational* adder tree of seven fp8_adder instances, all in one cycle. 85.7% of the delay sits in three chained adders. The structure – not the cells – is the violation.

4. Deliverable 4 – Optimized RTL implementation

(status: complete; two accepted edits, three instructive rejections)

Accepted – fp8_adder, logic restructure, latency +0. The model removed the sticky_a/sticky_b computation, replaced signed exponent arithmetic with unsigned compares, and moved the shift decisions out of the always block into continuous assigns. Its own stated reason: replacing dynamic shifters and sticky-bit calculation with a shallow case-based multiplexer tree and a fast priority encoder. **G1: EQUIVALENT, depth 1, complete, proof: combinational, 0.7 s.**

Rejected by G1b – fp4_dot_unit, pipeline, latency +4. See §6.1. This is the most interesting result in the project and it is a *rejection*.

Accepted – fp4_dot_stage, pipeline, latency +4. This is the edit that produces every number in §5, and it only exists because the G1b rejection above sent the loop one level up. fp4_dot_unit sits inside a rigid parent contract that four extra cycles break; fp4_dot_stage, the module that *instantiates* it, talks to the rest of the design over a valid/ready stream, where added latency is legal as long as no beat is invented, lost or reordered. So the same transform that is unprovable one level down is provable one level up – under a different theorem. fp4_dot_stage is checked by G1c, not G1: cycle-by-cycle equivalence is the wrong contract for an elastic interface, and G1 and G1b are skipped with that reason recorded in the history record rather than silently.

attempt	transform	latency	gate that spoke	result
0	pipeline	+4	G1c, 0.8 s, depth 16	REJECTED – property violated, counterexample trace returned
1	pipeline	+4	G1c 15.2 s, G2 PASS, G2c PASS	ACCEPTED

Attempt 0 cut the 31.66 ns multiply-add chain into four stages and tracked occupancy with a valid shift register, and G1c found a trace it could not satisfy in 0.8 s. Attempt 1 kept the same four-stage cut and changed one thing: the pipeline now advances while *any* stage is still occupied, so the tail of a burst drains instead of stalling in the last stages. That is the model's own stated fix, written against the returned counterexample. The +4 cycles are the “adding four stages of registers to the dot-product datapath” whose timing effect is measured on the repaired final tree in §5. The older opt run predates the parent repair and is retained only as audit history.

A second, independent run with the full gate stack armed. Everything above came from a loop in which G1b and the clock gate did not yet exist. To check that the added gates cost nothing on a clean edit, the loop was re-run from the untouched baseline with G1, G1b, G2 and G2c all active (artifacts/loop_g1b/). It converged in two model calls on the same module and, this time, on a different rewrite:

attempt	transform	latency	gate that spoke	result
0	logic_restructure	+0	G1, 0.4 s	REJECTED – in_a=0x87, in_b=0x53 gives 0x48, golden gives 0x53
1	logic_restructure	+0	G1 0.9 s, G2 PASS, G2c PASS	ACCEPTED

Attempt 0 is the loop working as designed rather than a failure: the model flattened the priority encoder *and* the sticky-bit logic in one step, got the sticky bit wrong, and the SAT solver produced a concrete input pair inside half a second. That counterexample went back into the next prompt; attempt 1 kept the flat encoder, replaced the variable shifter with a case of constant shifts, and left the sticky path alone. Both attempts are **proof: combinational, complete: true** – for a purely combinational module, depth 1 is not a bound, it is the whole theorem.

Two things are worth noting about the accepted record. G1b returned **NO_PARENT**-equivalent silence because the edit is latency-preserving, so the parent contract cannot be at issue – the gate costs 0 s on an edit that does not move latency, which is the common case. And G2c passed while still reporting both baseline clock bugs as **inherited from baseline**: the gate refuses the edit only for structures the *candidate* made worse, which is what keeps it usable on a design that is already dirty.

5. Deliverable 5 – Timing, frequency and PPA comparison

(status: complete for the repaired final RTL)

Both recorded runs use the same ORFS flow, PDK, SDC and floorplan and reach 6_final. The timing analysis reads the adjacent 6_final.spf and propagates clocks, matching ORFS's finish report.

Self-audit finding: the first headline omitted signoff parasitics

The first report re-read 6_final.odt alone. ORFS writes that database before RC extraction, then reads 6_final.spf and propagates clocks before producing 6_finish.rpt. The old positive WNS and zero TNS therefore matched neither ORFS run. We corrected the scripts and reran both sides. On the repaired final candidate, the extractor reports WNS –0.1659 ns and TNS –1.9487 ns, matching its own ORFS signoff report. The agreement is the sanity check.

The improvement remains large, but the chip is not fully timing-clean. The target clk_s5 closes with +1.4876 ns, while clk1 remains at –0.1659 ns and clk_s8 at –0.1494 ns. We caught our own optimistic measurement and corrected it against the flow's own signoff report.

Metric	Baseline	Final candidate	Delta
Design WNS	–25.2539 ns	–0.1659 ns	+25.0880 ns
Design TNS	–3384.4421 ns	–1.9487 ns	+3382.4934 ns
Hold WNS	–0.1329 ns	–0.0117 ns	+0.1212 ns
Hold TNS	–2.5864 ns	–0.0357 ns	+2.5507 ns
Area	1,746,071 μm^2	1,718,209 μm^2	–1.60%
Instances	480,545	475,618	–4,927 (–1.03%)
Nets	121,589	116,411	–5,178 (–4.26%)
Wirelength	6,901,284 μm	6,634,649 μm	–3.86%
DRC violations	0	0	–
Antenna violations	not extracted	1 net / 1 pin	open

At the nominal periods, 9 of 11 clock groups with reported paths meet setup timing after optimisation, up from 7. Four domains (clk2–clk5) report no path and are excluded: no path is an open constraint question, not a pass.

5.1 Maximum frequency

Fmax is measured by binary search over each clock's SDC period with extracted parasitics and propagated clocks. Both sides use the same –to 0.05 schedule.

Clock	Baseline Fmax	Final candidate Fmax	Change
clk_s5	25.65 MHz	89.09 MHz	+247.3% (3.47x)

Clock	Baseline Fmax	Final candidate Fmax	Change
clk1	86.81 MHz	95.90 MHz	+10.5%
clk_s1	154.68 MHz	126.74 MHz	-18.1%
clk_s2	117.51 MHz	123.52 MHz	+5.1%
clk_s3	95.90 MHz	100.80 MHz	+5.1%
clk_s4	136.99 MHz	112.26 MHz	-18.1%
clk_s8	86.81 MHz	95.90 MHz	+10.5%
clk_s1_gate	≥800.00 MHz (<i>floor</i>)	510.91 MHz	unresolved baseline delta
clk_s2_gate	473.93 MHz	450.86 MHz	-4.9%
clk_s4_gate	≥500.00 MHz (<i>floor</i>)	≥500.00 MHz (<i>floor</i>)	unresolved
clk_s5_gate	508.91 MHz	534.76 MHz	+5.1%
clk2-clk5	not measured	not measured	no timing paths

The headline supported by the corrected evidence is clk_s5 25.65 -> 89.09 MHz. Non-target clocks move in both directions; the report records those moves without claiming that the target edit caused each one. Generated-clock Fmax is not independent design headroom because each generated clock is derived from a master.

5.2 Physical-signoff limitations

The table is from an isolated ORFS run over artifacts/final_candidate/rtl, through 6_final. Detailed-route DRC is zero. The final antenna checker still reports one violating net and one pin, and the signoff timing report retains 36 setup and 6 hold violating endpoints. Power is not emitted by the comparison extractor. These are open signoff issues, not clean passes.

6. Deliverable 6 – Formal equivalence verification report

(status: complete)

The single frozen-to-final artifact is artifacts/final_candidate/toplevel_equiv.{md,json}. Its verdict is **equivalent modulo the declared stream latency** (artifact verdict: EQUIVALENT MODULO DECLARED STREAM LATENCY). This is not an ordinary clean top-level pass. It combines 52 byte-identical RTL files, a complete combinational proof for fp8_adder, and the six-property FP4 stream proof at depth 16 under the declared +4-cycle contract. An ordinary axi_lite_dot boundary miter correctly finds a cycle-16 counterexample because the result-valid pulse moves four cycles; the artifact preserves that counterexample instead of hiding it.

6.1 G1b – the result we did not expect

The loop pipelined fp4_dot_unit into four clean stages: the adder tree levelled, registers between levels, and – following a rule we had added after two earlier failures – **no resets on the added datapath registers**, since a pipeline register holds no state that has to be recovered and a reset on it makes the module disagree with its own unpipelined self for as long as reset is asserted.

G1 passed it: EQUIVALENT, latency +4, depth 6, proof: feed-forward – a *complete* proof, not a bounded one, because the module has no loop in its state graph. It took 578.8 s. The module is genuinely correct.

Its parent is not:

```
fp4_dot_unit fp4_dot_0(clk_2, rst_2, va, vb, dot_out);
assign fifo_data_write = dot_out;
assign w_en    = (!fifo_empty_1 && !fifo_empty_2 && !fifo_full);
assign r_en_1 = (!fifo_empty_1 && !fifo_empty_2 && !fifo_full);
```

The same cycle that pops the operands out of the two input FIFOs pushes dot_out into the result FIFO. That was correct while the dot unit was combinational. With four stages inside it, w_en now stores the product of operands popped four cycles earlier, or reset garbage. **Equivalent module, broken SoC.**

No module-scope equivalence checker can see this, because at module scope nothing is wrong. The check has to move up one level – and there it becomes easy again. Build the parent over the original child and over the transformed child, and miter the two parents against each other **with no delay wrapper**: the parent is not supposed to be late, it is supposed to be identical.

```
CONTRACT_BROKEN  fp4_dot_unit
parent axi_lite_dot      NOT_EQUIVALENT  18.8s
```

The loop now runs G1b whenever latency_delta != 0, and only then – a latency-preserving rewrite cannot violate a timing contract it did not touch. The failure feedback asks for a latency-preserving rewrite rather than another pipeline, because unlike every other gate, **a G1b failure is not a bug in the model's edit**. The edit is correct. The contract around it broke.

What happened when the gate ran live. All of the above was reconstructed from a run in which G1b did not yet exist. Re-running the loop with it armed (`artifacts/loop_g1b/`, iteration 1) reproduced the situation independently and then went one step further:

attempt	proposal	G1	G1b	outcome
0	pipeline, latency +3	EQUIVALENT	CONTRACT_BROKEN – <code>axi_lite_dot</code> NOT_EQUIVALENT, 20.2 s	REJECTED
1	–	–	–	REFUSED , with reason

The model chose three stages this time rather than four, so the specific edit was new; the contract broke exactly the same way. What it did with the feedback is the part worth reading:

The module is entirely combinational with no internal registers, ruling out `retime`. The 8-input floating-point adder tree is already balanced at the minimum possible depth of 3, and floating-point non-associativity prevents alternative tree structures under strict equivalence (`logic_restructure`). Since the parent module cannot tolerate any latency, we cannot apply `pipeline` to split the 31.6 ns path.

Every clause of that is correct, including the one that is easiest to get wrong: FP addition is not associative, so re-balancing the adder tree is *not* a logic restructure under bit-exact equivalence, and a model that tried it would have been rejected by G1 with a counterexample. Given a refused channel and a gate that had just told it precisely why its plan was invalid, the model declined to guess. A loop without that channel would have spent its remaining budget generating pipelines that G1b rejects one after another.

That run establishes the narrower result: **the 31.6 ns path cannot be closed by editing `fp4_dot_unit` while keeping its rigid parent contract**. Section 6.4 records the next step that succeeded: move the target to the FIFO-shaped `fp4_dot_stage` boundary, change the parent control with the child, and prove the resulting data stream under a declared +4-cycle contract. Ordinary parent equivalence remains false and is retained as part of the final artifact.

6.2 Completeness – three proof arguments, honestly labelled

`sat -seq N` proves equivalence for N cycles from reset. That is a bounded claim, and most of the time it is the honest one. Sometimes it is weaker than what is actually true, and saying so matters:

- **combinational** – neither side holds a flop, latch or memory, so agreeing for one cycle means agreeing forever. Depth 1 is a *complete* proof.
- **feed-forward** – no loop in the state graph, so latency + 2 cycles is complete.
- **bounded** – depth 12, reported as bounded.

Statelessness is asked of the *elaborated* design, not the source: grepping for `always @(posedge ...)` misses a latch accidentally inferred by an incomplete `always @(*)`, and a latch is state however the source reads. If Yosys fails, completeness is not claimed.

This is not bookkeeping. `fp8_adder` carries 85.7% of the worst path, is purely combinational, and is where the optimizer spends most of its attempts. Under the bounded rule its proofs cost 6.3 s and were labelled bounded; under this rule they cost 0.6 s and are **complete**.

6.3 Compositional RTL-vs-netlist equivalence

Separately from the transform gate, we check the synthesised netlist against the source RTL. A flat check gives up on exactly the modules that matter: `fp4_dot_unit` contains fifteen floating-point units, and flattened it times out at 900 s – even though all fifteen children were proved individually, minutes earlier, in seconds.

So we prove in dependency order and keep the results. A module that passes is emitted as a `(* blackbox *)` stub and read on **both** sides of every later check, cutting it out of the cone. This is compositional equivalence checking and it is what commercial EC does by default; it is the only reason EC scales.

```
fp4_mul      PASS   7.4s
fp8_adder    PASS  20.1s
fp4_dot_unit PASS   5.6s  boxed: 2   (flat: TIMEOUT 900s)
axi_lite_dot FAIL   6.5s  boxed: 1
```

`fp4_dot_unit` alone went from a 900 s timeout to 5.6 s. The standalone breakdown shows why: EQY's partitioned pass timed out at 900 s, its merged pass timed out at 900 s, and the plain Yosys miter – with the children boxed – proved it in **5.4 s**. Boxing the children is what made the module easy; paying the partitioner to rediscover that is waste, so a module with boxed children skips straight to the miter.

The soundness constraint, stated plainly: the chain is sound only because each child carries its own proof. A child that fails, times out, or is skipped is **not** boxed for its parents – it stays expanded, and its parents inherit the cost.

The first full 54-module sweep, and what it actually measured. 28/54 proved in 2417 s. The 26 failures fall into four groups, and two of them turned out to be our own tooling rather than anything about the design:

class	n	cause
No SAT model available for cell ...	12	keep_hierarchy – our bug, fixed
UNPROVEN (n cells) at depth 5	12	no match anchors in the netlist
TIMEOUT	1	multiplier_pipelined
yosys internal assert	1	soc_top

The keep_hierarchy bug. run_synth.py sets keep_hierarchy on every module precisely so the netlist stays hierarchical for this check – and flatten honours that attribute, so prep –flatten walked past every child and left it as an unflattened hierarchical cell. equiv_induct then met a cell it had no SAT model for and stopped with the same message a genuine blackbox produces:

ERROR: No SAT model available for cell mul_unit_ex_gate (multiplier_pipelined).

We had read that message as “compositional boxing does not work here” and documented it as such. It was the attribute. setattr –mod –unset keep_hierarchy before prep –flatten on both sides removes it. Where a *genuinely* boxed child causes the same error, the driver now drops the box and re-runs the module expanded with EQY's passes back on – slower, strictly stronger, and it converts a tool error into a verdict.

The 12 UNPROVEN modules are a different animal, and the root cause is worth stating precisely. ORFS synthesis strips internal net names: every wire in the netlist's timer module below the ports is _0000_, _0001_, ... equiv_make pairs gold and gate wires **by name**, so on these modules it finds exactly the port bits and nothing else – for timer, 33 \$equiv cells, all of them outputs, none internal. Induction with zero matched state points has nothing to anchor on, and it proved 0 of 33. Deeper induction does not help (–seq 20: still 33), nor does equiv_struct, nor forcing init values with setundef –init –zero. This is not a bounded-depth shortfall; it is a missing correspondence between the two state spaces, and the fix belongs on the synthesis side (a name policy that preserves register names) rather than in the checker. We report these as unproven rather than dressing them up.

Known limitation. asynchronous_fifo_gen is parameterised, so synthesis specialises it to \$paramod\$<hash>\asynchronous_fifo_gen and it never appears under its plain name. It is currently reported as not-found rather than proved, and we say so rather than counting it as a pass.

The second sweep, after the keep_hierarchy fix. 27 of 54 proved. The count went *down* by one, and that is the honest result rather than a disappointing one:

class	first sweep	second sweep
PROVED_EQUIVALENT	28	27
UNPROVEN (n cells) at depth 5	12	19
TIMEOUT after 900 s	1	6
No SAT model available for cell ...	12	1
yosys internal assert	1	1

Fixing our own bug did not buy proofs. It converted twelve tool errors into eleven honest verdicts – modules that now get as far as a real proof attempt and either run out of induction depth or run out of time – and left one, the id_memory_256x64 macro inside l1_i_cache_8kb, which is a genuine blackbox with no SAT model and never had one. That is what a fix to a measurement instrument is supposed to look like: the instrument stops lying, and the number it reports gets slightly worse.

The third sweep, and the two bugs it found in the sweep itself. We had written the 19 UNPROVEN modules off as a synthesis name-policy problem and moved on. Re-reading one module's logs closely enough turned up two defects above that, and neither of them was in the design.

The solver was never there. EQY's sby strategy shells out to an SMT engine by name. bitwuzla ships inside oss-cad-suite/bin, which is not on PATH unless the environment is sourced, and when the binary is missing the engine dies without a status, EQY reports the partition unproven, and the run degrades silently to the yosys fallback. No line anywhere says “no solver.” A full 54-module sweep was spent that way. eqy_netlist.py now puts that directory on PATH itself and refuses to start if the engine is still absent.

A cut-point file that was leaking. cutpoints.v carried a stub for every module proved so far, not only the children of the module under test, on the reasoning that a stub for a module the check never instantiates is inert. It is not: the stubs are read with read_verilog –lib, and prep –top prunes unused *real* modules while keeping unused blackboxes. Gold therefore reached EQY carrying nineteen modules it does not instantiate – alu, arbiter, fp4_mul, clk_gate – and gate, where those were real modules and got pruned, carried none of them:

combine: ERROR: Unmatched module exists in gold that does not exist in gate.
This should not happen. Please report this bug.

Both EQY passes died there, on every module, before a single solver call. The wrapper then fell through to the plain Yosys miter – which closes combinational logic and does not close sequential logic. That is the whole shape of the previous results, and it is a very clean shape once you look for it:

	proved	unproven
combinational modules	15	0
sequential modules	8	21

The split is on state, not on size and not on clocked-block count: eleven of the failures have a single `always @(posedge)` block, exactly like the eight that passed. A result that clean is rarely a solver limit. It was a module-list mismatch three steps upstream.

The same file also made every module's verdict a function of the entire pass/fail history above it, since the stub set grew as the sweep went. Two modules that passed one sweep failed the next for exactly that reason – `i_rom_32x256` and `id_memory_256x64_wrap`, same pass, same depth, different accumulated stub file. Boxing only children makes a module's check depend on its own subtree and nothing else, which is what the compositional soundness argument claimed in the first place.

Confirming the name-policy diagnosis instead of asserting it. With EQY actually running, `mem_axi_slave` still would not close, and its logs now say something precise. The merged pass reduces the module to exactly one partition, `mem_axi_slave.addr`, and returns equivalence unknown at depth 5 and again at depth 20. The identifier dumps say why:

```
GOLD: mem_axi_slave addr_reg w=31:0          <- a wire
GATE: mem_axi_slave addr_reg[9]$_SDFFE_PP0P_ c=$scopeinfo <- a scope marker
matched.ids: addr_reg -> 0 occurrences
```

The ORFS netlist has no wire named `addr_reg` at all. Flattening and ABC renamed every flop output, and the RTL name survives only as `$scopeinfo` metadata. EQY pairs gold to gate by wire name, finds no internal match point, and is left proving the module as a single partition with all of its state unmatched – and `addr_reg <= addr_reg + (1<<burst_size)` is an accumulator, so k-induction from an arbitrary initial state cannot close it at any depth. That is why depth 20 reads the same as depth 5.

So we tested the claim rather than repeating it: same RTL, same EQY, same depth, same solver, one netlist resynthesised with hierarchy and net names kept (`scripts/synth_named.py`).

gate netlist	addr_reg matched	verdict
ORFS 1_2_yosys.v	0	UNPROVEN (36 cells) at depth 5 and at depth 20
name-preserving resynth	1	PROVED_EQUIVALENT in 9.0 s

The diagnosis holds, and it is a statement about the netlist's name policy, not about the design or the checker. The ORFS netlist remains the PPA source of truth for §5; it is simply not usable as an equivalence-checking target, and the sweep now runs against a netlist built for that job.

That experiment settles one more thing. The partitioned pass reported **NOT_EQUIVALENT** on `mem_axi_slave` while the merged pass proved the same module equivalent – a worked example of the false counterexample that partitioning introduces and that the multi-pass ordering exists to absorb. A claimed counterexample is now recorded in `counterexample_claimed_by` rather than being promoted to the verdict or buried: it is a lead, not a disproof, and the field name says so.

The result. 36 of 54 modules proved equivalent in 3637 s, against 27 of 54 before tonight's two fixes. Nine modules moved from unproven to proved, and every one of them holds state, which is the class the diagnosis predicted: `mem_axi_slave`, `csr_regfile`, `register_file`, `dmem`, `timer`, `gpio_controller`, `l1_cache_axi_master`, `read_buffer_i_cache` and `instruction_decode_stage`. No module regressed.

class	sweep 2 (900 s)	sweep 4 (180 s)
PROVED_EQUIVALENT	27	36
TIMEOUT	6	10
UNPROVEN (n cells) at depth 5	19	7
No SAT model available for cell ...	1	0
yosys internal assert (soc_top)	1	1

The timeout was cut from 900 s to 180 s for this sweep, which needs justifying rather than assuming: against a name-preserving netlist a provable module proves in single-digit seconds – `alu` in 6.1 s, `mem_axi_slave` in 16.3 s, `l1_cache_axi_master`, the largest of them, in 75.2 s – so a long budget buys nothing on the modules that close and costs three full timeouts on each one that does not. The check is whether it cost any proof, and it did not: **every module that times out at 180 s also failed at 900 s**, as

a TIMEOUT or as UNPROVEN. Three of them (axi_lite_dot, branch_predictor, axi_lite_gpio) changed which failure they report, which is a label moving, not a proof lost.

The 18 that remain fall into three classes, and none of them is a claim about the design:

- **10 TIMEOUT** – the top of the tree (processor_top, execution_stage, instruction_fetch_stage, both L1 caches) plus the arithmetic-heavy leaves (fp4_dot_unit, multiplier_pipelined). These are the modules whose cones stay large because a child below them is unproven and therefore not boxed – the soundness constraint stated above, doing exactly what it says it will.
- **7 UNPROVEN at depth 5** – five of them (axi_lite_dma_config, dma_controller, axi_lite_timer, axi_lite_uart, axi_interconnect_2m_8s) hit ERROR: conflicting matches for gold bit \addr[0]: \addr[0] vs _0683_.A, where a gold bit is reachable in the gate both as a named wire and as a cell pin, so EQY declines to partition and the flat miter takes over. gate_nomatch_* and opt_clean -purge both fail to clear it; we report it as its own class rather than guess.
- **1 yosys internal assert** on soc_top – Assert 'count_id(wire->name) == 0' failed in kernel/rtlil.cc:2888, a tool bug, not a verdict.

Nothing in the sweep is NOT-EQUIVALENT. Every failure is UNPROVEN, TIMEOUT, or a tool error, and the difference matters: the checker never found a netlist that behaves differently from its RTL, it ran out of resources or anchors on half the tree. Reporting 36/54 as “two thirds of the design is verified” is the accurate claim; reporting it as “a third of the design is wrong” would be false, and reporting 54/54 by loosening the checker would be worse than either.

6.4 G1c – when equivalence itself is the wrong question

G1b ends its rejection by asking the model for a latency-preserving rewrite instead. On this path there is no such rewrite. The chain is 31.66 ns of FP multiply and three levels of FP add against a 12.5 ns period; no restructuring of combinational logic closes a 2.5x overrun, and FP8 addition is not associative, so the adder tree cannot even be re-balanced without changing the result. The only transform that closes this path adds cycles. G1b's advice, on this path, is advice to give up.

So the fix is not to make the child fit the parent's contract. It is to change the contract: make the parent elastic, and let it stall.

That reopens the verification question from the beginning, because a cycle-exact miter is now asking something that is not true and does not need to be. The module sits between three FIFOs. It is read from two and written to one, and **behind a FIFO the cycle a value arrives on is not observable**. What is observable – and what the SoC actually depends on – is the stream: the values written, and their order.

This is not a weaker obligation than equivalence. It is a different and stronger one, and we state it as six properties checked over a bounded unrolling, against the *original* combinational unit kept beside its own successor as a reference model:

	property	what it rules out
P1	!(w_en && full)	overflowing the result FIFO
P2a	!(w_en && occ == 0)	inventing a result that was never computed
P2b	occ <= DOT_LAT	losing one
P3	!w_en data_w == q_head	wrong value, or right values out of order
P4	stall_cnt <= DOT_LAT	stranding a finished result while there is room
–	r_en_1 == r_en_2	popping the two operand FIFOs out of step

P3 is the equivalence claim, restated where it is checkable. P4 is the one that earns the gate its place.

The measured result. We wrote two candidates. Both are four-stage pipelines, both cut at the same points, and they differ in one term of one expression – whether the pipeline advances only when new operands arrive, or also while any stage is still occupied:

```
wire adv = room && src;           // rejected
wire adv = room && (src || |vld); // accepted
```

The first passes every other gate in this flow. It is a correct pipeline of a correct function; nothing about it is unsound at module scope; G2 and G2c pass it. It also silently drops the last four results of every burst, because when the operand stream stops, nothing advances the results still in flight. G1c rejects it on P4.

6.4.1 The gate did not work, and why The first version of G1c could reject but could not accept, and the reason is worth reporting because it is the kind of failure that is easy to hide.

sat -seq N unrolls the design N times and hands the result to a SAT solver. Finding a counterexample is a satisfiability question and often lands early; proving there is none is an unsatisfiability question over the whole unrolled cone. That cone contained eight fp4_mul instances and a three-level fp8_adder tree, once per cycle of the bound. Measured, on the correct candidate:

Bound	Proof time, real arithmetic
4	5.6 s
6	362.2 s
10	no answer in 2400 s

And on the buggy candidate, the bound needed to see the bug:

Bound	Verdict, real arithmetic
4	STREAM_EQUIVALENT – bug not reachable
5	STREAM_EQUIVALENT – bug not reachable
10	PROPERTY_VIOLATED (P4), 34.3 s

Read those two tables together and the gate is broken. A missing drain term is not reachable until the pipeline has been filled and then starved, which takes more cycles than depth 5 provides – so shallow bounds accept the buggy candidate. Deep bounds catch it, but at a depth where the correct candidate cannot be proved at all. **There was no bound at which the gate both rejected what it should and accepted what it should.** A gate tuned to depth 4 because depth 4 is fast would have passed the bug and reported a proof.

6.4.2 Datapath abstraction Re-read the five properties: nothing is written when the sink is full, nothing is written that was never computed, nothing computed is lost, results leave in the order they entered, the pipeline drains. Not one of them mentions floating-point arithmetic. The arithmetic is what the solver spends its time on and none of it is what the properties are about.

So the leaves are replaced by surrogates – same ports, same widths, a cheap function instead of the real one. The device and the reference model resolve `fp4_mul` and `fp8_adder` from the same file set, so one substitution abstracts both sides identically and P3 keeps meaning what it meant: the value written is the value the reference produced from those operands, for whatever function the leaves compute. The surrogates are non-commutative on purpose – a commutative one would let a transform that reassoced the adder tree pass abstractly when it does not pass concretely.

Bound	Correct candidate	Buggy candidate
8	STREAM_EQUIVALENT, 0.7 s	PROPERTY_VIOLATED, 0.6 s
12	STREAM_EQUIVALENT, 3.1 s	PROPERTY_VIOLATED, 0.9 s
16	STREAM_EQUIVALENT, 22.5 s	PROPERTY_VIOLATED, 1.2 s

Accept and reject now work at the same bound, and the loop's default moved to 16 – deep enough to fill and then starve a four-stage pipeline, at a cost of seconds rather than an answer that never arrives.

What this cannot see, stated plainly. A candidate that changes the arithmetic itself passes G1c. That is a division of labour rather than a hole. A leaf is combinational and small, so G1 proves it directly and cheaply – `fp8_adder` in 1.0 s – while G1c proves the control that surrounds it. Neither gate alone is the argument; the pair is, and the same compositional stance is what §6.3 applies to RTL against netlist.

A second measured result, which surprised us. Run G1 – the cycle-exact miter, the gate this whole flow is built around – against the *correct* elastic candidate, and it returns `NOT_EQUIVALENT`. It is not malfunctioning. It is answering the question it was asked, and on an elastic interface that question has no useful answer: there is no fixed N for which the outputs match N cycles later, because the outputs are handshake signals whose timing depends on how full the destination FIFO happens to be.

So the loop does not run both. Which obligation applies is decided by the child's interface, not by the size of the latency change:

```
rigid interface  -> G1 (cycle-exact miter) + G1b (parent absorbs the shift)
elastic interface-> G1c (stream contract)    G1 and G1b are skipped
```

The skip is recorded in the history as `g1_skipped`, with its reason, so no reader of the log has to take our word for which gate ran.

`g1c_stream.applies()` decides this structurally, by port shape, and it claims nothing about a shape it has not been taught – a module that is not recognisably elastic returns `NOT_APPLICABLE` and falls through to G1, exactly as `clockcheck.py` reports `UN-RECOGNISED` rather than guessing at a clock structure it does not know. Recognising a shape is worth what the pattern is worth; failing to recognise one is a prompt to look, never a pass.

6.5 The gate's own bug – a bounded proof that proved nothing

Two of the five transforms in the catalog, `retime` and `duplicate_driver`, had never been exercised in either direction: zero proposals anywhere in the artifacts. That is the same blind spot that had already hidden one structural defect, so both were tested deliberately rather than assumed.

`retime` was tested on a real edit, not a mock: adder level 1 of `fp4_dot_unit` moved backwards across the stage-0 register boundary, so stage 0 holds four sums instead of eight products. Latency unchanged, stage count unchanged, register count 12 -> 8 – the definition of retiming. A second copy was deliberately mis-wired, pairing `y_2` with `y_4` instead of `y_3`.

depth	correct retime	mis-wired retime
4	EQUIVALENT, 2.7 s	EQUIVALENT, 2.6 s
5	TIMEOUT	NOT_EQUIVALENT, 5.4 s, counterexample at cycle 5
12	TIMEOUT, 300 s	–

The bold cell is the finding: **G1 accepted broken RTL**. `fp4_dot_unit`'s output sits four flops behind its inputs, so inside a four-cycle window both halves of the miter are still on their reset values. The solver proves them equal, truthfully, without ever having compared a value that depends on an input. The verdict flipped on the depth alone, and nothing in the record said so – it read **EQUIVALENT**, complete: false, proof: bounded, which is exactly what a real bounded proof reads like.

The fix is an observability probe, run before any bounded pass is reported. Golden is mitered against golden with one input bit inverted, at the same depth through the same passes. A model found means the flip reached an output inside the window, so the window sees the inputs. No model found means no input can, the bounded proof compared reset values only, and the verdict becomes **INCONCLUSIVE** rather than **EQUIVALENT**. Inversion is used rather than tying a bit low because a tie only differs when the solver happens to pick a 1 there, while an inversion differs on every vector. The probe runs on the bounded branch alone: depth 1 on a stateless module is a complete theorem already, and `latency + 2` on a feed-forward insertion derives its window from the latency it was given, so neither can be vacuous and neither pays for the check.

Two honest consequences follow. First, a correct retime of `fp4_dot_unit` is **not provable inside the budget** – TIMEOUT at depth 5 and at depth 12 – so the loop would reject it. That is the right side to fail on, and it is a real limit of this gate on this module, stated rather than hidden. Second, `duplicate_driver` was checked the same way on `branch_comp_decoder`, with the two opcode comparators duplicated per output bit: the correct copy **EQUIVALENT**, a copy given the wrong opcode **NOT_EQUIVALENT**, both in 0.0 s at depth 1 with a complete proof. Combinational duplication has no start-state and no window question at all, which is why it was the safe one – and why testing it was still the only way to know.

The general point is the one worth taking from this section. A formal gate is itself a piece of engineering that can be wrong, and the failure mode that matters is not the one that rejects good edits loudly – it is the one that accepts bad edits quietly, with a verdict string that looks exactly like a proof.

7. Deliverable 7 – Interactive demo

(status: complete; video pending)

`demo.py` at the repo root walks the whole flow in eight acts, in the order the system itself works in:

act	what it shows
1	seven critical-path targets ranked from the parasitic-aware baseline
2	the lever selector: which violations justify an RTL/model call
3	G1 – exact/bounded equivalence, with the completeness argument named
4	G1b – the parent contract, the rejection, and the model's refusal
5	G1c – the rejected dead pipeline and accepted six-property stream proof
6	G2 – CDC structure, inherited vs created
7	G2c – clock structure, and the two bugs in the baseline
8	G3 – final PPA and Fmax re-measured through the identical flow

Two modes, and the distinction matters for anyone reproducing this:

```
python3 demo.py          # replay: no API key, no network, seconds
python3 demo.py --live    # re-runs the real tools and the real model
python3 demo.py --act 4    # one act
```

Replay reads the recorded artifacts – `artifacts/loop/history.jsonl`, `artifacts/loop_g1b/history.jsonl`, and the final-candidate signoff JSON – and prints exactly what the run produced, including its failures. Nothing in replay mode is narrated from memory: Act 4's refusal text is read out of the run record. The repaired candidate's `6_final` metrics are wired into Act 8.

The replay was rehearsed successfully on 11 September; only the screen recording remains. --live re-executes the same acts against the tools, which takes minutes and may require a configured model key.

8. What we found in the benchmark that nobody was looking for

The clock-structure checker (clockcheck.py) exists because there is a class of bug that **neither** of our two guards can see. Equivalence cannot: a combinational clock mux and a flop-based one compute the same function of sel, and a SAT miter has no notion of a pulse too short to clock a flop. STA cannot: OpenSTA is told clk_out is a clock, believes it, and reports against a clean idealised waveform – the tool is not wrong, nobody asked it whether that waveform is producible.

Run over the 55-module baseline it returns exactly two findings, and no false positives:

```
GLITCHY_MUX    clk_div_mux.clk_out
               selected combinational among clk, clk_div2, clk_div4, clk_div8
GLITCHY_GATE    clk_gate.clk_out
               clk_out = clk_in & clk_en, no low-phase latch on the enable
```

Both are real bugs in the benchmark RTL as written. sel changing while the selected clock is high truncates that pulse; a pulse below the library's minimum width is not a slow edge, it is a flop capturing metastable data anywhere in the fanout. And clk_en moving while clk_in is high chops the gated pulse – the safe form latches the enable on the low phase so it can only change while the clock is already low.

clk_div_mux is slicer target P003 with 100% path share: the optimizer reaches it.

Getting to zero false positives across 55 modules took two rules beyond the name. A clock net must be **one bit wide** – soc_top holds clkcfg_rdata, clk_config_reg and clk_en_reg, all clock-configuration buses, and a name-only rule reported the AXI read mux over them as a glitchy clock mux. And a name ending _en, _sel, _gate, _valid or _req is what *gates* a clock, not a clock.

The checker also recognises the fix, or it could only ever say no. Rewriting clk_gate as a standard ICG flips it to SAFE_ICG, and all three regression directions behave correctly.

In the loop it runs in regression mode with the same stance G2 takes on crossings: **a transform may inherit a glitchy structure; it may not create one.** The baseline already contains two of these, and a gate that failed every candidate for a bug the candidate did not introduce would be switched off within a day.

8.1 The warning we could not prove, and what we did with it

G2 proves a one-bit crossing has two flops and no logic between them. On a bus it cannot, and it says so – the accepted run above carries exactly two G2 warnings, both on asynchronous_fifo_gen's pointers. Two flops per bit is *not* a synchronizer for a bus: the bits resolve independently, so a value in flight can be sampled as a combination that never existed on the source side. What makes a bus crossing safe is the encoding around it, and an encoding is not a structure a depth-counting checker can recognise.

cdc_classify.py classifies the encoding, in two layers with a hard line between them. Structural evidence is sound and needs no model: a gray signature, a dual-clock FIFO instantiation, a req/ack pair. Anything else goes to the model and is recorded **basis: model, advisory: true**, carrying **no verdict field at all** – it gates nothing. That split is the inverse of how the model is used everywhere else here: the LLM proposes and formal tools judge, so a model asked to *judge* a CDC produces a hypothesis for a human, never a proof. A CDC bug waved through by a model is precisely the failure this project argues against.

On this design both warnings resolve structurally:

```
asynchronous_fifo_gen.counter_1_gray  clk_1 -> clk_2  gray [structural] high
    counter_1_gray = counter_1^{1'b0, counter_1[logofdepth:1]}
asynchronous_fifo_gen.counter_2_gray  clk_2 -> clk_1  gray [structural] high
```

Getting there took one correction worth recording. The first version knew only the textbook spelling $x \wedge (x \gg 1)$ and returned unclear on both pointers – which is worse than a miss, because an unclassified crossing is exactly what gets handed to a model to guess about. This design writes the same encoding as a concatenation, $x \wedge \{1'b0, x[msb:1]\}$. The matcher now takes both spellings, and matches only against the driver of *the signal in question* rather than against any gray conversion anywhere in the module – a FIFO holds two counters and crediting the wrong one is the mistake the check exists to prevent.

9. Design decisions and dead ends

The full record is bench/DECISIONS.md – 19 numbered decisions, each with the evidence that forced it. The ones worth reading:

- **The counterexample that said nothing.** Truncating sat's table to fit a prompt sent the model 17 lines of reset state and none of the failure. Two identical failures were caused by feedback that contained no information.
 - **opt -fast does not run opt_merge.** Both halves of a miter come from the same RTL, so every untouched cell appears twice, identically, driven by the same inputs. `opt_merge -share_all` collapses those pairs before the solver sees them – on the pipelined dot product that deletes eight multipliers and all of level 1. Without it, the first attempt at that proof spent 1847 s and returned nothing.
 - **opt -full aborts on miters holding \$mem,** which is every FIFO here – so `opt -fast` plus an explicit `opt_merge`, not `opt -full`.
 - **memory_map is mandatory.** Left as an array, storage stays a `$mem_v2` cell and the solver stops with *No SAT model available*.
 - **A 503 must not discard proven work.** A provider outage once took down a run holding an accepted, proved edit. `Back-endUnavailable` is now distinct from a refusal: a refusal is an answer, an outage is the absence of one, and the loop records it and stops early keeping everything already accepted.
 - **We first ran EQY against an unusable netlist** – ORFS flattened hierarchy and ABC renamed the register anchors that EQY needs. The named synthesis netlist restored those anchors and raised the result from 27/54 to 36/54; the ORFS netlist remains the PPA source of truth.
-

10. Honest limitations

1. The candidate PnR comparison in §5 is two proven transformations on one benchmark design. It demonstrates a working flow; it is not a broad benchmark study.
2. G1's bounded proofs are bounded. Where the module allows a complete argument we make it and label it; where it does not, depth 12 is what we claim.
3. G1b proves the *parent* unchanged. The final parent deliberately changes to absorb latency, so ordinary equivalence fails correctly; G1c supplies a bounded depth-16 stream-contract proof instead.
4. `clockcheck.py` is structural. It recognises the shapes it was taught; `UNRECOGNISED` is a prompt to look, not a verdict.
5. Multi-bit CDC crossings are reported as warnings, not proven. G2 cannot verify that a bus crossing is gray-coded or handshake-protected.
6. `asynchronous_fifo_gen` is not covered by the netlist EC sweep (§6.3).